

? Frequently Asked Questions (FAQ)

Common questions and troubleshooting for the E-Commerce Automotive Store template.

Table of Contents

1. [Installation & Setup](#)
 2. [Database Issues](#)
 3. [Stripe & Payments](#)
 4. [Deployment](#)
 5. [Customization](#)
 6. [Performance](#)
 7. [Security](#)
 8. [Features & Functionality](#)
-

Installation & Setup

Q: What are the minimum system requirements?

A:

- Node.js 18.0 or higher
- npm 9.0 or higher
- PostgreSQL 14.0 or higher
- At least 2GB RAM
- 500MB free disk space

Q: I'm getting "Module not found" errors. What should I do?

A: Try these solutions in order:

1. Reinstall dependencies:

```
bash
rm -rf node_modules package-lock.json
npm install
```

2. Clear Next.js cache:

```
bash
rm -rf .next
npm run dev
```

3. Check Node.js version:

```
bash
node --version # Should be 18.0+
```

4. Use correct package manager:

```
bash
# If you see pnpm-lock.yaml or yarn.lock, use those instead
```

```
pnpm install # or
yarn install
```

Q: How do I know if my installation is successful?

A: Verify these checkpoints:

- ✓ No errors during `npm install`
- ✓ Development server starts without errors
- ✓ Homepage loads at `http://localhost:3000`
- ✓ No console errors in browser DevTools
- ✓ Products display on homepage
- ✓ Can navigate between pages
- ✓ Database connection successful

Q: Port 3000 is already in use. What do I do?

A: Several solutions:

Option 1 - Kill the process:

```
# macOS/Linux
lsof -ti:3000 | xargs kill -9

# Windows
netstat -ano | findstr :3000
taskkill /PID [PID_NUMBER] /F
```

Option 2 - Use a different port:

```
PORT=3001 npm run dev
```

Option 3 - Find what's using the port:

```
lsof -i :3000 # macOS/Linux
netstat -ano | findstr :3000 # Windows
```

Q: Can I use Yarn or pnpm instead of npm?

A: Yes! The template works with all package managers:

```
# Yarn
yarn install
yarn dev

# pnpm
pnpm install
pnpm dev

# Bun (experimental)
bun install
bun dev
```

Just make sure to use the same package manager throughout the project.

Database Issues

Q: I'm getting "Database connection refused" errors

A: Follow these troubleshooting steps:

1. **Check if PostgreSQL is running:**

```
```bash
macOS
brew services list | grep postgresql

Linux
sudo systemctl status postgresql

Windows
Check Services app for PostgreSQL service
```
```

1. **Start PostgreSQL if stopped:**

```
```bash
macOS
brew services start postgresql

Linux
sudo systemctl start postgresql

Windows
Start via Services app or pgAdmin
```
```

1. **Verify connection string:**

```
env
# Format: postgresql://USERNAME:PASSWORD@HOST:PORT/DATABASE
DATABASE_URL="postgresql://postgres:mypassword@localhost:5432/automotive_store"
```

2. **Test connection manually:**

```
bash
psql -U postgres -d automotive_store -c "SELECT 1"
```

Q: How do I reset my database?

A: ⚠ **Warning: This deletes all data!**

```
# Drop and recreate database
psql -U postgres -c "DROP DATABASE automotive_store;"
psql -U postgres -c "CREATE DATABASE automotive_store;"

# Run migrations again
psql -U postgres -d automotive_store -f scripts/schema.sql

# Reseed data (optional)
psql -U postgres -d automotive_store -f scripts/seed.sql
```

Q: Where is my database data stored?

A: PostgreSQL data location:

- **macOS (Homebrew):** `/usr/local/var/postgresql@14/`
- **Linux:** `/var/lib/postgresql/14/main/`
- **Windows:** `C:\Program Files\PostgreSQL\14\data\`

You typically don't need to access these directly.

Q: Can I use MySQL instead of PostgreSQL?

A: The template is built for PostgreSQL, but you can adapt it:

1. Change database driver to `mysql2`
2. Update connection string format
3. Modify SQL queries for MySQL syntax differences
4. Update schema file for MySQL data types

Recommendation: Stick with PostgreSQL for easiest setup.

Q: How do I backup my database?

A:

Backup:

```
pg_dump -U postgres automotive_store > backup.sql
```

Restore:

```
psql -U postgres automotive_store < backup.sql
```

Automated backups: Set up a cron job (Linux/macOS) or Task Scheduler (Windows).

Stripe & Payments

Q: Can I use PayPal instead of Stripe?

A: The template uses Stripe exclusively. To add PayPal:

1. You'll need to integrate PayPal SDK separately
2. Create PayPal checkout flow alongside Stripe
3. Handle PayPal webhooks
4. This requires significant development work

Recommendation: Stripe supports multiple payment methods (cards, Apple Pay, Google Pay) which covers most customers.

Q: Why am I getting “No such customer” errors in Stripe?

A: Common causes:

1. **Using wrong API keys:**

- Test customers don't exist in live mode
- Live customers don't exist in test mode
- Solution: Match keys to environment

2. **Customer ID doesn't exist:**

- Check customer ID in Stripe Dashboard
- Verify database has correct customer ID saved

3. **Using customer from different Stripe account:**

- Customers are account-specific
- Can't use customer IDs across accounts

Q: Test payments work, but live payments fail. Why?

A: Checklist:

- ☐ Using live API keys (not test keys)
- ☐ Stripe account fully activated
- ☐ Bank account added and verified
- ☐ Business details completed
- ☐ Production webhook configured
- ☐ Using real credit card (not test cards)

Q: How do I refund an order?

A:

Via Stripe Dashboard:

1. Go to Stripe Dashboard → Payments
2. Find the payment
3. Click “Refund payment”
4. Enter amount (full or partial)
5. Confirm refund

Via API (if implementing in your app):

```
const refund = await stripe.refunds.create({
  payment_intent: 'pi_...',
  amount: 5000, // Optional: partial refund in cents
})
```

Q: Can customers save their cards for future purchases?

A: Yes! Enable in checkout session:

```
const session = await stripe.checkout.sessions.create({
  payment_intent_data: {
    setup_future_usage: 'on_session',
  },
  // ... other params
})
```

Then implement customer portal for saved cards.

Q: Webhook is not receiving events. What's wrong?

A: Troubleshooting steps:

1. Local development:

```
bash
# Ensure Stripe CLI is running
stripe listen --forward-to localhost:3000/api/checkout/webhook
```

2. Production:

- Verify webhook URL in Stripe Dashboard
- Check webhook endpoint is publicly accessible
- Test with "Send test webhook" in Stripe
- Check server logs for incoming requests

3. Common issues:

- Server not running
- Incorrect webhook URL
- Firewall blocking Stripe IPs
- Wrong webhook secret in environment variables

Q: How do I support multiple currencies?

A:

1. Store prices in multiple currencies in database

2. Detect customer location:

```
typescript
const currency = detectCurrency(customerCountry)
```

3. Display prices in customer's currency:

```
typescript
formatPrice(amount, currency)
```

4. Create checkout with correct currency:

```
typescript
const session = await stripe.checkout.sessions.create({
  line_items: [{
    price_data: {
      currency: customerCurrency, // 'usd', 'aud', 'eur', etc.
      // ...
    }
  }]
})
```

Deployment

Q: Which hosting platform should I use?

A: Recommendations:

Best overall: Vercel

- ☒ Built for Next.js
- ☒ Easy setup
- ☒ Automatic deployments
- ☒ Great free tier
- ☒ Serverless (not ideal for long-running tasks)

Great for beginners: Railway

- ☒ Simple interface
- ☒ Includes database
- ☒ Affordable
- ☒ Smaller free tier

For control: VPS (DigitalOcean, Linode)

- ☒ Full control
- ☒ Fixed pricing
- ☒ More setup required
- ☒ You manage everything

Q: My deployment is failing with “Build failed” error

A: Common causes and fixes:

1. **TypeScript errors:**

```
bash
# Check locally first
npm run build
```

Fix any TypeScript errors before deploying.

2. **Environment variables missing:**

- Ensure all required variables set in platform
- Check for typos in variable names

3. **Dependencies missing:**

- Check all imports have corresponding packages in `package.json`
- Run `npm install` locally to test

4. **Build timeout:**

- Optimize build process
- Reduce dependencies
- Upgrade hosting plan

Q: Site deployed successfully but shows “Internal Server Error”

A: Check these:

1. **Database connection:**

- Verify `DATABASE_URL` is correct

- Check database is running and accessible
- Test connection from deployment platform

2. **Environment variables:**

- All variables copied to production?
- No typos in variable names?
- Values correct for production?

3. **Server logs:**

- Check deployment platform logs
- Look for specific error messages
- Check for missing secrets

Q: How do I set up a custom domain?

A: See [DEPLOYMENT_GUIDE.md](#) (./DEPLOYMENT_GUIDE.md#custom-domain-setup) for detailed instructions.

Quick summary:

1. Add domain in hosting platform
2. Update DNS records at domain registrar
3. Wait for DNS propagation (1-48 hours)
4. HTTPS automatically configured

Q: How do I enable HTTPS?

A:

Vercel/Railway:  Automatic - HTTPS enabled by default

Self-hosted: Configure SSL certificate:

```
# Using Certbot (Let's Encrypt)
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Customization

Q: How do I change the site colors?

A: Edit `tailwind.config.js` :


```

module.exports = {
  theme: {
    extend: {
      colors: {
        primary: {
          500: '#YOUR_PRIMARY_COLOR',
        },
        secondary: {
          500: '#YOUR_SECONDARY_COLOR',
        },
      },
    },
  },
},
}

```

See [CUSTOMIZATION_GUIDE.md](#) (./CUSTOMIZATION_GUIDE.md#color-scheme--theme) for details.

Q: How do I replace the logo?

A:

1. Prepare your logo:

- Format: SVG (preferred) or PNG
- Dimensions: ~200x50px recommended

2. Add to project:

```
public/images/logo.svg
```

3. Update in code:

```

tsx
// components/Header.tsx
<Image src="/images/logo.svg" alt="Your Store" width={200} height={50} />

```

See [CUSTOMIZATION_GUIDE.md](#) (./CUSTOMIZATION_GUIDE.md#logo--favicon) for details.

Q: Can I use this template for non-automotive products?

A: Absolutely! It's a general e-commerce template. To adapt:

1. Replace product categories
2. Update product data in database
3. Change site copy and descriptions
4. Customize branding and colors
5. Update SEO metadata

The core functionality works for any product type.

Q: How do I add a blog section?

A:

1. Create blog page structure:

```

app/blog/page.tsx      # Blog list
app/blog/[slug]/page.tsx # Individual post

```

2. Add database table:

```
sql
```

```
CREATE TABLE blog_posts (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255),
  slug VARCHAR(255) UNIQUE,
  content TEXT,
  author VARCHAR(255),
  published_date TIMESTAMP,
  image_url TEXT
);
```

3. Create blog components

4. Add to navigation

Q: How do I change the currency from AUD to USD?

A:

1. Update price display:

```
typescript
// lib/utils.ts
export function formatPrice(amount: number) {
  return new Intl.NumberFormat('en-US', {
    style: 'currency',
    currency: 'USD', // Changed from 'AUD'
  }).format(amount)
}
```

2. Update Stripe currency:

```
typescript
// Stripe checkout session
price_data: {
  currency: 'usd', // Changed from 'aud'
}
```

Performance

Q: My site is loading slowly. How do I improve performance?

A: Optimization checklist:

Images:

- ☒ Use Next.js `<Image>` component
- ☒ Compress images (TinyPNG, Squoosh)
- ☒ Use modern formats (WebP, AVIF)
- ☒ Lazy load images
- ☒ Add image CDN (Cloudinary, Imgix)

Code:

- ☒ Remove unused dependencies
- ☒ Code split with dynamic imports

- ☒ Minimize client-side JavaScript
- ☒ Use React Server Components

Database:

- ☒ Add indexes on frequently queried columns
- ☒ Optimize slow queries
- ☒ Implement caching (Redis)
- ☒ Use connection pooling

Hosting:

- ☒ Enable CDN (Vercel Edge Network)
- ☒ Choose server region close to users
- ☒ Use database in same region as server

Q: How do I enable caching?

A:

Next.js App Router (Recommended):

```
// Automatic caching
export const revalidate = 3600 // Revalidate every hour

export async function getProducts() {
  const res = await fetch('https://api.example.com/products', {
    next: { revalidate: 3600 } // Cache for 1 hour
  })
  return res.json()
}
```

Redis caching:

```
import Redis from 'ioredis'
const redis = new Redis(process.env.REDIS_URL)

export async function getCachedProducts() {
  const cached = await redis.get('products')
  if (cached) return JSON.parse(cached)

  const products = await fetchProducts()
  await redis.set('products', JSON.stringify(products), 'EX', 3600)
  return products
}
```

Q: How can I monitor site performance?

A: Use these tools:

Built-in:

- Vercel Analytics (if using Vercel)
- Next.js Speed Insights
- Lighthouse (Chrome DevTools)

Third-party:

- Google Analytics
- GTmetrix

- WebPageTest
- New Relic
- Datadog

Security

Q: Is this template secure for production use?

A: The template follows security best practices:

- ✓ Environment variables for secrets
- ✓ HTTPS in production
- ✓ Stripe webhook signature verification
- ✓ SQL injection prevention (parameterized queries)
- ✓ XSS protection (React escapes by default)
- ✓ CSRF protection (Next.js built-in)
- ✓ Rate limiting (recommended to add)

Additional security measures to implement:

- Add rate limiting (express-rate-limit)
- Implement CAPTCHA for forms
- Set up security headers (next-secure-headers)
- Regular dependency updates
- Monitor for vulnerabilities (Snyk, Dependabot)

Q: How do I handle sensitive data securely?

A:

1. Use environment variables:

```
env
# Never commit to Git
DATABASE_URL=...
STRIPE_SECRET_KEY=...
```

2. Add to .gitignore:

```
.env
.env.local
.env.production
```

3. Use secret management services:

- Vercel: Built-in encrypted secrets
- Railway: Environment variables
- AWS: Secrets Manager
- HashiCorp: Vault

4. Never log sensitive data:

```
```typescript
// ❌ DON'T
console.log('User password:', password)
console.log('Card number:', cardNumber)
```

```
// ✅ DO
console.log('Authentication attempt for user:', userId)
console.log('Payment processed:', paymentId)
...
```

## Q: How do I protect against SQL injection?

**A:** Use parameterized queries:

```
// ❌ VULNERABLE - Don't do this!
db.query(`SELECT * FROM users WHERE email = '${email}'`)

// ✅ SAFE - Always use parameterized queries
db.query('SELECT * FROM users WHERE email = $1', [email])

// With Prisma
prisma.user.findUnique({ where: { email } })
```

The template uses parameterized queries throughout.

## Q: Should I enable CORS? How?

**A:** Only if your frontend and backend are on different domains.

```
// next.config.js
module.exports = {
 async headers() {
 return [
 {
 source: '/api/:path*',
 headers: [
 { key: 'Access-Control-Allow-Origin', value: 'https://yourdomain.com' },
 { key: 'Access-Control-Allow-Methods', value:
'GET,POST,PUT,DELETE,OPTIONS' },
],
 },
],
 },
}
```

**Security tip:** Specify exact origins, never use `*` in production.

## Features & Functionality

### Q: Does this support user accounts and authentication?

**A:** Yes! The template includes NextAuth.js for authentication:

- ✅ Email/password login
- ✅ Social login (Google, GitHub configurable)
- ✅ Session management
- ✅ Protected routes
- ✅ User profiles

## Q: Is there an admin panel included?

**A:** Yes! Located at `/admin` :

Features:

- View all orders
- Manage products
- View customers
- Dashboard with analytics
- Order status management

**Admin access:** Set up admin users in database.

## Q: Does it support multiple languages?

**A:** Yes, i18n support is included using `next-i18next` :

```
// To add a new language:
// 1. Add translation files
public/locales/es/common.json // Spanish
public/locales/fr/common.json // French

// 2. Configure in next-i18next.config.js
module.exports = {
 i18n: {
 locales: ['en', 'es', 'fr'],
 defaultLocale: 'en',
 },
}
```

## Q: Can customers track their orders?

**A:** Yes! Features included:

- Order confirmation emails with order number
- Order history in user dashboard
- Order status updates
- Email notifications for status changes

## Q: Does it support discount codes and coupons?

**A:** Basic coupon support is included. For advanced features:

1. Create coupons in Stripe Dashboard
2. Apply in checkout session:

```
typescript
```

```
discounts: [{ coupon: 'SUMMER2024' }]
```

For complex discount rules, you may need custom implementation.

## Q: Is shipping calculation included?

**A:** Basic shipping is included. You can:

1. Set flat rate shipping
2. Integrate with shipping APIs (e.g., ShipStation, EasyPost)
3. Use Stripe's shipping rates feature

## Q: Can I sell digital products?

**A:** Yes! The template supports both physical and digital products:

For digital products:

- Skip shipping address collection
  - Provide download links after purchase
  - Send download link in confirmation email
- 

## Still Have Questions?

---

**Check these resources:**

- **Setup Guide:** [BUYER\\_SETUP\\_GUIDE.md](#) (./BUYER\_SETUP\_GUIDE.md)
- **Customization:** [CUSTOMIZATION\\_GUIDE.md](#) (./CUSTOMIZATION\_GUIDE.md)
- **Deployment:** [DEPLOYMENT\\_GUIDE.md](#) (./DEPLOYMENT\_GUIDE.md)
- **Stripe:** [STRIPE\\_SETUP\\_GUIDE.md](#) (./STRIPE\_SETUP\_GUIDE.md)
- **Troubleshooting:** [troubleshooting.md](#) (./troubleshooting.md)
- **Developer Guide:** [developer\\_guide.md](#) (./developer\_guide.md)

**Need more help?**

- Check the official Next.js documentation
  - Review Stripe documentation
  - Search Stack Overflow
  - Check GitHub Issues
- 

**Last Updated:** November 2024